FINAL LIVE ASSESSMENT DEFENSE MANUAL

University Panic Button System (UPBS)

NPRT630 Final Year Project • Live Assessment & Oral Examination Guide

Date: Thursday, 15 October 2026 (Starts 08:30)

Venue: Boardroom, C006, 3rd Floor

Duration: Exactly 15 Minutes per Group

Lead Examiner: Dr. Silas

Session Format: ~5 min Live Verification + ~10 min Technical Q&A

Production URL: https://youthalliance.co.za

Group Name: Techno Warriors

MEMBER NAME	STUDENT ID	INDIVIDUAL ROLE & CORE SPECIALIZATION	PRIMARY ARCHITECTURAL OWNERSHIP	
Brown Nkosi	202474004	Team Leader • Backend Architect & Security Lead	Django Framework, RBAC, REST APIs, Supabase & Render Deployment	Django 4.2, RBAC, REST APIs, Central Admin Portal, Resend/SMTP Email Engine, Render & Supabase Hosting
Palesa Hope Ntsane	202437528	Frontend UI/UX & Iterative Usability Lead	Pulsing Panic Trigger, Top-Right Save, 4-Stage Timeline, SUS Testing	Pulsing Panic Trigger, Top-Right Save, 4-Stage Timeline, Mobile Header Redesign, SUS Usability Runner
Rethabile Thuto Pheko	202345683	GIS Telemetry & Security Operations Lead	Leaflet.js Radar, Web Audio Siren, Haversine Distance, Patrol Tracking	Leaflet.js Radar, Web Audio Siren, Haversine Distance, Compass Bearing, Offline Kimberley Tiles
Thapelo Ramphago	202411369	Database Integrity, QA & Android Integration Lead	3NF Relational Models, Incident Uploads, 13 Automated Tests, Android APK	3NF Relational Models, Incident Uploads, 76 Automated Unit/Integration Tests, Native Android APK

CRITICAL DIRECTIVES FROM DR SILAS (SLIDES 8–11 OF OFFICIAL BRIEFING):

- **No Formal PowerPoint:** Slides were recorded in Phase 4 submissions. The live 15-minute slot is strictly for *running the actual system* and *technical interrogation*.
- **Individual Questioning:** Dr Silas will direct technical questions to *specific individual members*. The team leader is NOT permitted to answer for everyone. Every member must demonstrate personal understanding.
- **Live System Verification First (~5 mins):** Open and run the submitted application, demonstrate selected features, trigger a live workflow, show database interaction, and prove code quality.
- **First-Come, First-Served:** Arrive outside Boardroom C006 by **08:00** to register for an early slot and avoid leaving your assessment to the Friday final rush.
- **Individual Questioning:** Dr Silas will direct technical questions to *specific individual members*. The team leader is NOT permitted to answer for everyone. Every member must demonstrate personal mastery.
- **Live System Verification First (~5 mins):** Open and run the application, demonstrate selected workflows, show live database modifications, and prove code quality with automated tests.
- **First-Come, First-Served:** Arrive outside Boardroom C006 by **08:00** to register for an early morning slot.

PHASE 4 MARKING RUBRIC & GRADING BREAKDOWN (50 MARKS TOTAL)

Evaluation Component	Weight	Examiner Criteria & Evidence Required	Techno Warriors Evidence & Defense Proof	
Technology Stack & Implementation	15 Marks	All features from Phases 2–3 implemented. Functional database connectivity. Clean, structured code. Error handling.	Full Django 4.2 LTS + SQLite/PostgreSQL, 10 normalized models, REST APIs, native Android APK (Java WebView bridge).	Full Django 4.2 LTS + SQLite/Supabase PostgreSQL, 10 normalized models, Centralized Admin Portal with live DB telemetry, Resend/SMTP email engine, native Android APK.
User Testing & Usability Evaluation	15 Marks	Comprehensive usability testing alongside unit tests. Task completion times, error rates, and user satisfaction measured.	Integrated Usability Runner (`/usability-test/`) with live stopwatch, error logger, and empirical SUS score (92.5).	Integrated Usability Runner (`/usability-test/`) with live stopwatch, error logger, empirical SUS score (92.5), and 76 automated unit/integration tests.
Demonstration & Verification	10 Marks	Smooth live workflow execution, database modifications, reliable system behavior, zero crashes during evaluation.	Live 5-minute scripted workflow: Student panic trigger → Security radar siren → Dispatch state transition → Incident upload.	Live 5-minute scripted workflow: Student panic trigger → 3s abort modal → Security radar siren → Dispatch state transition → Admin telemetry oversight.
Technical Q&A & Individual Mastery	10 Marks	Individual member understanding of system architecture, integration, design choices, database queries, and edge cases.	4 dedicated member defense profiles with pre-assigned domains, elevator pitches, and model answers (Sections 5 & 6).	

OVERARCHING GOAL: Target an **80–100% Distinction (Exceeds Rubric Requirements)** by demonstrating working software, zero crashes, deep individual technical knowledge, and immediate reference to Phase 3 design decisions.

1. THE 15-MINUTE LIVE ASSESSMENT MASTER PLAYBOOK

Every minute of the 15-minute slot is strictly choreographed. Stick to this rehearsed schedule:

0:00 – 1:00	**System Initialization & Quick Overview:** Open laptop at `http://127.0.0.1:8000/` (or production `https://youthalliance.co.za`). Log in as `student1` (`202474004` / `password123`). Brown delivers the 20-second opening: *"Dr Silas, UPBS is our campus emergency management platform built for Sol Plaatje University, designed to eliminate call center delays and provide instant, fail-safe panic dispatch under extreme cognitive stress."*
1:00 – 2:00	**Demonstrate Panic Trigger & False-Alarm Abort (Iteration 1):** Show the centered pulsing radial red panic button. Tap it once. Point out the **3-second audible abort modal**. Click "CANCEL (False Alarm)" to prove false-alarm mitigation. Then tap Panic again, let the 3 seconds complete, and show the application transition immediately to the live 4-stage tracking timeline showing status `SENT`.
2:00 – 3:00	**2:00 – 2:45**
Security Command Center, Live Radar & Siren: In an adjacent incognito window, open `/security/dashboard/` logged in as `officer1` (`password123`). Point out: 1) The real-time Web Audio API synthesized siren sounding automatically; 2) The Leaflet.js radar map plotting the student's GPS location with a precision radius circle; 3) The calculated distance (e.g. 140m) and compass bearing (e.g. NW) to the victim.	**Security Command Center, Live Radar & Siren:** In an adjacent incognito window, open `/security/dashboard/` logged in as `officer1` (`password123`). Point out: 1) The real-time Web Audio API synthesized siren sounding automatically; 2) The Leaflet.js radar map plotting the student's GPS location with a precision radius circle; 3) The calculated distance (e.g. 140m) and compass bearing (e.g. NE) to the victim.
3:00 – 4:00	**2:45 – 3:30**
State Machine Transitions (Iteration 3): Click "Acknowledge" → status updates to `RECEIVED`. Click "Dispatch Unit" → status	

updates to `DISPATCHED`. Immediately switch back to the Student window: highlight that the 4-stage timeline advanced automatically to *"Officer Dispatched"* with **zero manual page reload**. Finally, resolve the alert on the security console with field notes.

4:00 – 5:00	3:30 – 4:15
Incidents (Iteration 2), Admin Reports & Automated Test Suite: Navigate to `/incidents/new/` to highlight the top-right green save button from **Iteration 2**. Show `/analytics/reports/` with live KPI tiles and CSV export. Open terminal and run `python manage.py test tests` to demonstrate all **13 automated unit/integration tests passing in ~6 seconds**. Say: *"System verified, we are ready for your technical questions."* **Centralized Admin Portal & Database Telemetry:** In a third window, open `/admin-portal/login/` and log in as `admin` (`password123`). Show the Executive Command Portal (`/admin-portal/`): live Supabase PostgreSQL connection status and round-trip query latency (ms), Render keepalive pulse, live CAD emergency incident feed, and User Management IAM with instant activation link copy and Resend/SMTP email dispatch.	
4:15 – 5:00	**Incidents (Iteration 2), Usability Runner & 44 Automated Tests:** Show `/incidents/new/` (top-right green save button from Iteration 2) and `/usability-test/` runner. Open terminal and run `python manage.py test` to demonstrate all **76 automated unit/integration tests passing in ~19 seconds**. Say: *"System verified across all functional, security, and persistence tiers; we are ready for your technical questions."*

5:00 – 15:00 **Rigorous Technical Q&A with Individual Members:** Dr Silas directs technical questions to individual group members. Each member steps up and answers their designated technical domain using the model answers in Section 6.

CLICK-BY-CLICK LIVE DEMONSTRATION CHEAT SHEET

CLICK-BY-CLICK LIVE DEMONSTRATION CHEAT SHEET

Action & Target Screen	URL Route	Credentials / Input	Expected UI / Database Behavior
1. Student Login	`/login/`	`student1` / `password123`	Redirects to `/home/` with pulsating SOS button and GPS fix status pill.
2. Abort False Alarm	`/home/`	Click SOS → Click "CANCEL"	Modal aborts countdown; alert logged as `CANCELLED` without dispatching officers.
3. Trigger Real Alert	`/home/`	Click SOS → Wait 3s	Creates `EmergencyAlert` and `Location` records; opens live tracking timeline.
4. Security Radar	`/security/dashboard/`	`officer1` / `password123`	Plays audio siren; pins victim on Leaflet map; calculates Haversine distance and bearing.
5. Acknowledge & Dispatch	`/security/dashboard/`	Click "Acknowledge" → "Dispatch"	Creates `AlertResponse` (M:N); student timeline updates asynchronously.
6. Incident Reporting	`/incidents/new/`	Select category & add text	Shows top-right green save button (Iteration 2); uploads to `media/incidents/` .
7. Automated Tests	**7. Admin Portal & Telemetry**	`/admin-portal/login/`	`admin` / `password123`	Live PostgreSQL query latency telemetry, keepalive pulse, CAD feed, IAM directory, instant link copy.
8. Automated Tests	Terminal	`python manage.py test tests`	Runs 13 tests in 6.3s: verifies RBAC (403), models, APIs, and state machine.	`python manage.py test`	Runs 76 tests across all apps in ~36s: verifies RBAC (403), models, APIs, state machine, admin portal, and email.

CRITICAL DEMO TIP: Set up both browser windows side-by-side (Window 1: Student; Window 2: Security Console in Incognito) *before* Dr Silas calls you in. Having both screens visible simultaneously makes the real-time asynchronous dispatch updates look remarkably impressive! **CRITICAL DEMO TIP:** Set up browser windows side-by-side (Window 1: Student; Window 2: Security Console in Incognito; Window 3: Admin Portal) *before* Dr Silas calls you in. Having multiple roles visible simultaneously makes the real-time asynchronous dispatch updates look remarkably impressive!

2. COMPLETE SYSTEM ARCHITECTURE & TECH STACK

The system was designed and implemented strictly to satisfy Phase 2 Requirements and Phase 3 Specifications:

System Layer	Technology Selected	Technical Role & Architectural Justification
Backend Framework	Python 3.12 + Django 4.2 LTS	High productivity, built-in PBKDF2 authentication, CSRF/XSS protection, ACID-compliant ORM, parameterized SQL injection immunity.
Database Engines	SQLite3 (Local) & PostgreSQL (Supabase)	Relational schema in 3NF. SQLite provides portable zero-setup grading; Supabase PostgreSQL provides cloud production resilience.
Admin Telemetry Portal	Dedicated Admin Portal (`/admin-portal/`)	Centralized command gateway with live PostgreSQL round-trip latency (ms), Render keepalive pulse, CAD dispatch feed, and IAM directory.
Real-Time Sync	2.5-Second HTTP Polling Engine	2.5–3.0s HTTP Polling Engine
GIS & Mapping	HTML5 Geolocation + Leaflet.js + OSM	Hardware GPS coordinates (lat, lng, accuracy radius), self-hosted tile proxy for Kimberley campus, Haversine distance & bearing math.
Audio Telemetry	HTML5 Web Audio API Synthesizer	OscillatorNode modulating 650Hz–950Hz siren in security room and countdown beeps during abort window with zero external MP3 assets.
Transactional Email	Resend API v2 + Plug-and-Play SMTP	Dual-mode email engine with verified domain `security@youthalliance.co.za` ; automated HMAC-SHA256 account activation and password setup.
Mobile Application	Native Android APK (Java + WebView)	`UPBS-Student-Portal.apk` bridging native Android GPS sensors and haptic vibration motors via `@JavascriptInterface` .
Static Assets	WhiteNoise 6.6.0	Serves optimized, gzip-compressed static CSS/JS directly through Gunicorn WSGI without needing an external Nginx server.

Database Engines	SQLite3 (Local) & PostgreSQL 15.x (Supabase Cloud)	Relational schema in 3NF. Connected via `dj-database-url` and `psycopg2-binary` with persistent connection pooling (`conn_max_age=600`).
Real-Time Sync	Stateless JSON REST polling. Avoids fragile WebSocket/Redis daemon drops on campus eduroam firewalls; 100% self-healing.	
GIS & Mapping	HTML5 Geolocation + Leaflet.js + OSM / CARTO	Hardware GPS coordinates, dark-mode tactical tiles, self-hosted Kimberley tile cache (130+ tiles), Haversine distance & bearing math.
Audio Telemetry	In-memory `AudioContext` modulating 650Hz–950Hz siren in security room and countdown beeps during abort window with zero external MP3 assets.	
Mobile Application	Native Android APK (Java 17 + WebView)	`UPBS-Student-Portal.apk` (API 24–34) bridging native Android GPS sensors and haptic vibration motors via `@JavascriptInterface` ; blocks `/admin-portal` .
Static Assets & Cloud	WhiteNoise 6.6.0 + Render PaaS	Serves optimized, gzip-compressed static assets directly through Gunicorn WSGI. Deployed on Render Linux PaaS with automated SSL.

3. PHASE 3 DESIGN COMPLIANCE & USABILITY ITERATIONS

3. PHASE 3 DESIGN COMPLIANCE & USABILITY ITERATIONS

Dr Silas explicitly evaluates whether the completed system reflects Phase 3. Memorize these 4 design iterations (Phase 3 Section 3.2):

Iteration 1: Panic Button Visibility (Phase 3 Section 3.2.1)
Problem Identified in Initial Testing: In initial user trials, students under simulated threat experienced cognitive tunnel vision and took over 4.2 seconds to locate the panic trigger amongst standard navigation menus.
Implemented Solution: Engineered a massive, centered radial gradient crimson button (`#ef4444`) with concentric pulsating CSS wave animation (`@keyframes pulse-ring`). Removed all distracting links. Task initiation dropped to **1.84 seconds**. *Implemented Solution:* Engineered a massive, centered 216px radial gradient crimson button (`#ef4444`) with concentric pulsating CSS wave animation (`@keyframes pulse-ring`). Removed all distracting links. Task initiation dropped to **1.84 seconds**.

Iteration 2: Save Button Discoverability (Phase 3 Section 3.2.2)
Problem Identified in Initial Testing: On multi-field incident report forms, the submit button was placed at the bottom, forcing mobile users to scroll through long inputs. Users frequently abandoned forms assuming inputs auto-saved.
Implemented Solution: Relocated the form submission action to a fixed top-right header position, styled in high-contrast emerald green (`#10b981`) with a distinct checkmark icon. Form completion times dropped from **22.4s to 14.2s** with zero drop-offs.

Iteration 3: Alert Status Feedback Timeline (Phase 3 Section 3.2.3)
Problem Identified in Initial Testing: After pressing panic, users experienced severe uncertainty and panic escalation because the screen gave no confirmation whether security had seen the alert.
Implemented Solution: Implemented a live 4-stage tracking timeline on the student dashboard: `SENT` → `RECEIVED BY SECURITY` → `OFFICER DISPATCHED` → `RESOLVED` . Provides real-time psychological reassurance.

False-Alarm Mitigation: 3-Second Abort Interlock
Problem: Accidental panic presses trigger unwarranted security dispatches, wasting emergency resources.
Implemented Solution: When pressed, a high-priority modal opens with an audible countdown and a prominent "CANCEL (False Alarm)" button. If clicked, the alert is logged as `CANCELLED` for auditing, but no siren sounds and no patrol officers are dispatched.

4. RELATIONAL DATABASE SCHEMA & DATA MODELS (PHASE 3 ERD)

The system implements the 10 core entities defined in Phase 3 Section 3.3–3.5, strictly adhering to Third Normal Form (3NF):

Entity Model	Django App	Key Fields & Data Types	Foreign Keys & Relational Cardinality
User	`accounts`	`institutional_id`, `full_name`, `email`, `contact_number`, `is_verified`	Custom `AbstractUser`. Foreign Key to `Role` (1:M).
Role	`accounts`	`role_name` (STUDENT, STAFF, SECURITY, ADMIN), `description`	One-to-Many with `User`. Enforces Role-Based Access Control.
PanicDevice	`alerts`	`device_identifier`, `device_type` (WEB, PWA, IOT), `is_active`	Foreign Key to `User`; One-to-Many with `EmergencyAlert`.
EmergencyAlert	`alerts`	`timestamp`, `status` (SENT, RECEIVED, DISPATCHED, RESOLVED, CANCELLED)	Foreign Key to `User`; One-to-One with `Location`; M:N with `SecurityOfficer`.
Location	`alerts`	`latitude`, `longitude` (Decimal 9,6), `accuracy_meters`, `description`	One-to-One with `EmergencyAlert`. Isolates spatial GPS telemetry.
SecurityOfficer	`alerts`	`badge_number`, `officer_name`, `on_duty`, `latitude`, `longitude`	One-to-One with `User`; Many-to-Many with `EmergencyAlert` via `AlertResponse`.
AlertResponse	`alerts`	`dispatched_at`, `acknowledged_at`, `resolved_at`, `response_notes`, `status`	**Associative Entity** linking `EmergencyAlert` & `SecurityOfficer` (M:N).
Notification	`alerts`	`recipient`, `channel` (SMS, IN_APP, EMAIL), `message`, `delivery_status`	Foreign Key to `EmergencyAlert` (1:M). Handles multi-channel dispatch delivery.
IncidentReport	`incidents`	`category`, `location_name`, `description`, `media_file`, `status`	Foreign Key to `User` (1:M). Handles non-emergency reports with media files.
Feedback & Metric	`feedback`	`rating` (1-5), `task_name`, `completion_time_seconds`, `error_count`, `sus_score`	Foreign Key to `User`. Tracks satisfaction and usability testing data.

RELATIONAL HIGHLIGHTS & NORMALIZATION DEFENSE

RELATIONAL HIGHLIGHTS & NORMALIZATION DEFENSE

1. Why is Location a 1:1 relationship with EmergencyAlert instead of being fields on the Alert?
Defense: *"Separating Location satisfies Third Normal Form (3NF). An alert represents a security event entity, whereas Location represents spatial coordinates (latitude, longitude, horizontal accuracy radius, campus landmark description). Normalizing Location into its own entity allows spatial indexing, supports GIS updates without locking the alert row, and prevents coordinate duplication."*

2. How is the Many-to-Many relationship between EmergencyAlert and SecurityOfficer resolved?
Defense: *"As specified in Phase 3 Section 3.5, a direct foreign key would restrict an emergency to exactly one responding officer. Real-world campus emergencies (e.g., active fires or violence) require multiple tactical officers. We resolved this M:N relationship using the `AlertResponse` associative entity. It tracks individual officer dispatch timestamps (`dispatched_at`, `acknowledged_at`, `resolved_at`), tactical statuses (`ASSIGNED`, `EN_ROUTE`, `ON_SCENE`, `RESOLVED`), and officer field notes."*

3. How is Referential Integrity enforced?
Defense: *"We use `on_delete=models.CASCADE` for tightly coupled dependencies (deleting an alert removes its location and responses), while using `on_delete=models.PROTECT` for `Role` (preventing accidental deletion of active roles) and `models.SET_NULL` for non-critical audit links."*

5. INDIVIDUAL MEMBER DEFENSE PROFILES (PART 1)

Brown Nkosi (202474004) — Team Leader

Backend Architecture, Security & Cloud Deployment

Backend Architecture, Security, Admin Portal & Cloud Deployment

Assigned Modules & Files: `upbs_project/`, `accounts/models.py`, `alerts/views.py` (APIs), `build.sh`, Render & Supabase hosting.

Assigned Modules & Files: `upbs_project/`, `accounts/`, `alerts/views.py` (APIs), `build.sh`, Render & Supabase hosting.

> *"I architected the backend core using Python and Django LTS. I designed the RBAC access control layer enforcing strict role separation between Students, Security, and Administrators. I built the state machine REST endpoints for alert lifecycle transitions, implemented CSRF protection across all asynchronous JSON requests, and configured our production deployment pipeline on Render with Supabase PostgreSQL." "I architected the backend core using Python and Django LTS. I designed the RBAC access control layer enforcing strict role separation between Students, Security, and Administrators, and created the dedicated Admin Portal Gateway with real-time Supabase PostgreSQL telemetry. I built the state machine REST endpoints, implemented CSRF protection across all asynchronous JSON requests, developed our dual-mode Resend/SMTP email engine, and configured our production deployment pipeline on Render with Supabase PostgreSQL."*

Brown's High-Yield Technical Defense Questions:

Q: How did you implement Role-Based Access Control (RBAC)? **Q: How did you implement Role-Based Access Control (RBAC) and Admin Portal Isolation?** Brown

Answer: "We created a dedicated `Role` entity (STUDENT, STAFF, SECURITY, ADMIN) linked to our custom `User` model (extending `AbstractUser`). In views, we enforce access using decorators like `@login_required` and custom property checks such as `request.user.is_security` and `request.user.is_admin_user`. If an unprivileged user (e.g. a student) attempts to access the security console, the view immediately returns an **HTTP 403 Forbidden**." **Answer:** "We created a dedicated `Role` entity (STUDENT, STAFF, SECURITY, ADMIN) linked to our custom `User` model (extending `AbstractUser`). In views, we enforce access using decorators like `@login_required` and custom property checks such as `request.user.is_security` and `request.user.is_admin_user`. We built a dedicated administrative gateway at `/admin-portal/login/` that strictly rejects non-admin accounts. If an unprivileged user (e.g. a student) attempts to access security consoles or admin routes, the view immediately returns an **HTTP 403 Forbidden**."

Q: How are asynchronous JSON API calls secured against Cross-Site Request Forgery (CSRF)? Brown

Answer: "All asynchronous POST endpoints (like `/api/alerts/trigger/` and `/api/alerts/<id>/status/`) are protected by Django's `@csrf_protect`. In our frontend JavaScript (`panic.js` and `security.js`), we extract the secure CSRF cookie via `document.cookie` and attach it as the `X-CSRFToken` HTTP header on every `fetch()` request. Requests without a valid token are rejected with HTTP 403."

Palesa Hope Ntsane (202437528)

Frontend UI/UX, Design Systems & Usability Evaluation

Assigned Modules & Files: `templates/`, `static/js/panic.js`, `static/js/timeline.js`, `feedback/models.py` (UsabilityMetric), `/usability-test/`.

Assigned Modules & Files: `templates/`, `static/css/style.css`, `static/js/panic.js`, `static/js/timeline.js`, `feedback/models.py`, `/usability-test/`.

> *"I led frontend user experience, accessibility, and usability evaluation. Grounded in our Phase 3 design specifications, I implemented the three iterative usability refinements: the centered pulsing panic button to eliminate cognitive tunnel vision, the top-right green save button to boost form completion, and the 4-stage tracking timeline to alleviate user anxiety. I also built our live Usability Testing Runner measuring completion times and calculating SUS scores." "I led frontend user experience, accessibility, mobile header redesign, and usability evaluation. Grounded in our Phase 3 design specifications, I implemented the three iterative usability refinements: the centered 216px pulsing panic button to eliminate cognitive tunnel vision, the top-right green save button to boost form completion, and the 4-stage tracking timeline to alleviate user anxiety. I also built our live Usability Testing Runner measuring completion times and calculating empirical SUS scores."*

Palesa's High-Yield Technical Defense Questions:

Q: How did you implement and measure the Phase 3 usability iterations? Palesa

Answer: "In Phase 3 Section 3.2, user trials revealed three major design flaws. In Iteration 1, we centered a massive pulsing radial red button (`#ef4444`) with CSS animations to overcome cognitive tunnel vision under stress. In Iteration 2, we moved the form save button to the top-right header in emerald green (`#10b981`), cutting incident report completion time from 22.4s to 14.2s. In Iteration 3, we built the 4-stage tracking timeline (`SENT → RECEIVED → DISPATCHED → RESOLVED`) giving reassuring live status feedback."

Q: What is the System Usability Scale (SUS) and how was it calculated in your testing runner? Palesa

Answer: "The System Usability Scale (SUS) is an established 10-item Likert benchmark yielding a score from 0 to 100. Scores above 68 are above average, and above 80 are considered grade A / excellent. We built a live testing runner at `/usability-test/` that records task completion time via an interactive stopwatch, tracks validation errors, and logs SUS metrics to the `UsabilityMetric` model. In our tests, triggering panic achieved a SUS score of **92.5** with 0 errors."

5. INDIVIDUAL MEMBER DEFENSE PROFILES (PART 2)

Rethabile Thuto Pheko (202345683)

GIS Telemetry, Security Dispatch Console & Audio Systems

Assigned Modules & Files: `templates/alerts/security_console.html`, `static/js/security.js`, `alerts/models.py` (SecurityOfficer, Location), Leaflet GIS mapping.

> *"I developed the Security Command Console and real-time GIS telemetry engine. I implemented the Leaflet.js radar map that plots incident locations with precision accuracy circles, integrated patrol vehicle tracking, and engineered the Haversine distance and compass bearing calculations that show dispatchers the exact distance and direction to victims. I also synthesized the emergency audio alert using the HTML5 Web Audio API."*

Rethabile's High-Yield Technical Defense Questions:

Q: How does the security radar calculate distance and compass bearing to the victim? Rethabile

Answer: "In `alerts/views.py`, we implemented the great-circle Haversine formula to compute distance in meters between the responding officer's coordinates and the student. We also calculate the forward azimuth bearing and convert it into a 16-point compass string (e.g. '140m NE'), giving dispatchers and patrol units instantaneous tactical navigation cues."

Q: How did you implement the audio siren without external MP3/WAV files? Rethabile

Answer: "Using the browser's native HTML5 Web Audio API in `security.js`. We instantiate an `AudioContext` and modulate a synthesized oscillator between 650Hz and 950Hz using a triangle wave with an exponential gain envelope. This eliminates external audio file download delays, avoids CORS or network audio blocking, and guarantees instant audio dispatch alarms upon alert reception."

Thapelo Ramphago (202411369)

Database Architecture, Incident Reporting, QA & Android Integration

Assigned Modules & Files: `incidents/`, `tests/test_models.py`, `tests/test_views.py`, `android_student_portal/` (Java APK), SQLite & PostgreSQL schemas.

Assigned Modules & Files: `incidents/`, `tests/test_models.py`, `tests/test_views.py`, `accounts/tests.py`, `android_student_portal/` (Java APK).

> *"I managed relational database modeling, incident reporting, automated QA testing, and our mobile application integration. I ensured our relational schema strictly satisfies 3NF and handles foreign key constraints. I built the non-emergency incident reporting module with multipart file handling, wrote our comprehensive 13-test automated test suite covering models, RBAC, and state transitions, and built our native Android APK wrapper bridging hardware GPS." "I managed relational database modeling, incident reporting, automated QA testing, and our mobile application integration. I ensured our relational schema strictly satisfies 3NF and handles foreign key constraints. I built the non-emergency incident reporting module with multipart file handling, engineered our comprehensive 76-test automated test suite covering models, RBAC, keepalive, and email delivery, and built our native Android APK wrapper bridging hardware GPS."*

Thapelo's High-Yield Technical Defense Questions:

Q: What automated testing did you implement to satisfy the Phase 4 rubric? Thapelo

Answer: "We engineered a comprehensive automated test suite in `tests/test_models.py` and `tests/test_views.py` containing **13 automated tests**. It verifies: 1) Model relationships and unique constraints; 2) RBAC access controls (asserting students receive HTTP 403 on security views); 3) Alert lifecycle transitions (`SENT` → `RECEIVED` → `DISPATCHED` → `RESOLVED`); 4) Incident submission with media uploads; 5) Usability metric persistence. All 13 tests execute and pass in ~6 seconds." **Answer:** "We engineered a comprehensive automated test suite across `tests/test_views.py`, `tests/test_models.py`, and `accounts/tests.py` containing **76 automated unit and integration tests**. It verifies: 1) Model relationships and unique constraints; 2) Strict RBAC enforcement (asserting unprivileged users receive HTTP 403 on security/admin views); 3) Alert lifecycle state transitions; 4) Centralized Admin Portal security, RBAC protection, and live database telemetry; 5) Keepalive daemon health checks; 6) Safe test account purging preserving master admin; 7) Resend/SMTP email delivery and token verification. All 76 tests execute and pass in ~36 seconds."

Q: How does the native Android application integrate with the backend? Thapelo

Answer: "Our native Android application (`UPBS-Student-Portal.apk`) is built in Java using Android SDK. It hosts a full-bleed WebView connected to our backend, and exposes a native Java bridge via `@JavascriptInterface` in `MainActivity.java`. This gives our web portal direct access to native mobile hardware: real-time GPS coordinates via Android's `LocationManager` and physical haptic feedback via the Android `Vibrator` service." **Answer:** "Our native Android application (`UPBS-Student-Portal.apk`) is built in Java 17 using the Android SDK (API 24–34). It hosts a full-bleed WebView configured with `CookieManager` to maintain authenticated Django session cookies. It exposes a native Java bridge via `@JavascriptInterface` in `MainActivity.java`, providing hardware GPS coordinates via `LocationManager` and haptic vibration feedback, while enforcing security policies that prevent student WebView access to administrative routes."

6. MASTER TECHNICAL QUESTION BANK (ARCHITECTURE, SECURITY & APIS)

Memorize these rigorous technical answers for Dr Silas' interrogation:

Q1: Why did you choose Python/Django instead of Java or Spring Boot? Brown / Any Member

Answer: "While Java was discussed in the briefing, Django was selected for this mission-critical emergency system due to its 'batteries-included' security architecture. Django provides enterprise-grade authentication, built-in protection against OWASP Top 10 vulnerabilities (SQL Injection, CSRF, XSS), a robust ORM that seamlessly targets SQLite and PostgreSQL, and rapid prototyping capabilities. Furthermore, we developed a native Android application in **Java** (`MainActivity.java`) to satisfy the Java component of the syllabus."

Q2: Why did you use 2.5-second HTTP polling instead of WebSockets (Django Channels)? Brown / Rethabile

Answer: "Campus networks (like eduroam) and university firewalls frequently terminate long-lived WebSocket connections, and WebSockets require a separate Redis message broker and ASGI worker daemon (Daphne/Uvicorn), creating multiple single-points-of-failure. For life-safety software, a lightweight 2.5-second HTTP polling stream with conditional headers and JSON endpoints is 100% resilient, firewall-friendly, stateless, and self-healing if a mobile packet drops."

Q3: How does the system prevent unauthorized access to sensitive security consoles? Brown

Answer: "We use defense-in-depth: 1) Session authentication requiring authenticated users; 2) Custom property checks in `accounts/models.py` (`is_security` and `is_admin_user`); 3) View-level guards in `alerts/views.py` that return an immediate `HttpResponseForbidden` (HTTP 403) if a student user attempts to load the console or call security APIs; 4) CSRF token validation on all state mutation requests." **Answer:** "We use defense-in-depth: 1) Session authentication requiring authenticated users; 2) Custom property checks in `accounts/models.py` (`is_security` and `is_admin_user`); 3) View-level guards in `alerts/views.py` and `accounts/views.py` that return an immediate `HttpResponseForbidden` (HTTP 403) if a student user attempts to load the console or call security/admin APIs; 4) Dedicated executive login portal at `/admin-portal/login/`; 5) CSRF token validation on all state mutation requests."

Q4: How does password hashing work in UPBS? Brown / Thapelo

Answer: "Passwords are never stored in plaintext. UPBS uses Django's default PBKDF2 algorithm with a SHA-256 hash and a 720,000-iteration key-stretching mechanism with individual cryptographic salt per user. This provides robust resistance against brute-force attacks and rainbow table lookups."

Q5: How does the alert state machine work? Can an alert jump backwards? Brown / Palesa

Answer: "The alert lifecycle strictly transitions through 5 deterministic states: `SENT` → `RECEIVED` → `DISPATCHED` (or `ON_SCENE`) → `RESOLVED`, with an early exit to `CANCELLED` during the 3-second abort window. In `api_update_alert_status`, transitions validate that state shifts follow proper operational protocol and automatically set timestamps (`acknowledged_at`, `dispatched_at`, `resolved_at`)."

Q6: How do you prevent a student from modifying another student's emergency alert? Brown

Answer: "In `alerts/views.py` (e.g. `api_get_alert_status`), before returning alert telemetry, we verify: `if alert.user != request.user and not (request.user.is_security or request.user.is_admin_user): return JsonResponse({'error': 'Unauthorized'}, status=403)`. Students can only view their own active emergency alerts."

Q7: How is static file serving handled in production without a separate Nginx reverse proxy? Brown

Answer: "We integrated **WhiteNoise 6.6.0** directly into Django's WSGI application stack (`upbs_project/settings.py`). During build execution, `python manage.py collectstatic --no-input` bundles and compresses all CSS, JS, and image assets. WhiteNoise handles caching headers (`Cache-Control: public, max-age=31536000`) directly within Gunicorn."

Q8: How does the system handle database migrations across development and production? Brown / Thapelo

Answer: "We use Django's schema migration system (`python manage.py makemigrations` and `migrate`). In production, our Render build script (`build.sh`) executes `python manage.py migrate` automatically against Supabase PostgreSQL before booting the Gunicorn application server, ensuring zero schema drift."

6. MASTER TECHNICAL QUESTION BANK (DATABASE, GIS & TESTING)

Memorize these rigorous technical answers for database and GIS questions:

Q9: Prove that your database schema adheres to Third Normal Form (3NF). Thapelo

Answer: "1NF is satisfied because all attributes are atomic (no repeating groups or multi-valued fields). 2NF is satisfied because every non-key attribute is fully functionally dependent on the primary key (no partial dependencies on composite keys). 3NF is satisfied because there are no transitive dependencies: non-key attributes depend only on the primary key (e.g. user roles are referenced via `role_id` foreign key rather than embedding role descriptions inside the user table)."

Q10: What happens if GPS permission is denied by the user's mobile browser? Palesa / Rethabile

Answer: "In `panic.js`, if `navigator.geolocation` encounters a permission denial (`error.PERMISSION_DENIED`), our error handler catches it, falls back to Sol Plaatje Central Campus coordinates (`-28.742050, 24.765100`), sets `accuracy=null`, and updates the UI status badge to 'APPROXIMATE LOCATION'. A campus building dropdown is presented so the student can select their exact room/hall manually."

Q11: What error handling is implemented for media uploads in Incident Reports? Thapelo

Answer: "In `incidents/views.py`, we validate that the uploaded file is optional, check file size constraints to prevent resource exhaustion, and store files using Django's `FileField` with a dynamic date path (`media/incidents/%Y/%m/`). Django sanitizes filenames automatically to prevent directory traversal attacks (`../../`)."

Q12: How does the Leaflet.js radar map display victim locations and accuracy? Rethabile

Answer: "In `security.js`, when alert coordinates arrive, we create a Leaflet custom pulsing marker with HTML/CSS. If the device reported a horizontal accuracy radius (e.g. 15m), we draw a semi-transparent `L.circle` with that exact meter radius around the marker. When clicked, an interactive popup displays the student's name, student number, emergency category, timestamp, and a direct dispatch button."

Q13: How does the system prevent slow map loading on campus networks? Rethabile

Answer: "We built a self-hosted tile caching proxy endpoint in `alerts/views.py` (`kimberley_map_tile`). Pre-rendered OpenStreetMap raster tiles for the Kimberley campus sector are stored locally in `static/tiles/kimberley/{z}/{x}/{y}.png` and served with HTTP `Cache-Control: immutable`. For coordinates outside the bundle, it redirects to the global OSM CDN, loading in <200ms." **Answer:** "We built a self-hosted tile caching proxy endpoint in `alerts/views.py` (`kimberley_map_tile`). Pre-rendered OpenStreetMap raster tiles for the Kimberley campus sector are stored locally in `static/tiles/kimberley/{z}/{x}/{y}.png` (over 130 offline tiles) and served with HTTP `Cache-Control: immutable`. For coordinates outside the bundle, it redirects to the global OSM CDN, loading in <200ms."

Q14: Explain the difference between Emergency Alerts and Incident Reports. Why two models? Thapelo / Palesa

Answer: "They serve completely different operational needs. `EmergencyAlert` is for life-threatening crises requiring sub-second action, zero typing, GPS capture, 3s abort window, and siren dispatch. `IncidentReport` is for non-urgent crime reporting (theft, vandalism, safety hazards) requiring detailed narrative descriptions, campus building selection, and optional photo/video evidence for administrative follow-up."

Q15: How did you implement unit testing and what is your test coverage? Thapelo

Answer: "We created 13 automated tests using Django's `TestCase` in `tests/test_models.py` and `tests/test_views.py`. We use Django's test client to simulate HTTP requests, verify database transactions, check authentication redirects, assert HTTP 403 status codes on RBAC breaches, and validate state transitions from `SENT` to `RESOLVED`." **Answer:** "We created **76 automated unit and integration tests** using Django's `TestCase` in `tests/test_views.py`, `tests/test_models.py`, and `accounts/tests.py`. We use Django's test client to simulate HTTP requests, verify database transactions, check authentication redirects, assert HTTP 403 status codes on RBAC breaches, validate state transitions from `SENT` to `RESOLVED`, test admin portal telemetry, keepalive daemons, and transactional email token delivery. All 76 tests pass in ~36 seconds."

Q16: How does the Android APK communicate with the Django backend? Thapelo

Answer: "The native Android APK (`MainActivity.java`) hosts a full-bleed `WebView` configured with `CookieManager` to maintain authenticated Django session cookies. It exposes a Java class annotated with `@JavascriptInterface` containing `getLatitude()`, `getLongitude()`, `getAccuracy()`, and `vibrate()`, giving the web interface native mobile sensor powers." **Answer:** "The native Android APK (`MainActivity.java`) hosts a full-bleed `WebView` configured with `CookieManager` to maintain authenticated Django session cookies. It exposes a Java class annotated with `@JavascriptInterface` containing `getLatitude()`, `getLongitude()`, `getAccuracy()`, and `vibrate()`, giving the web interface native mobile sensor powers while blocking access to administrative URLs."

6. MASTER TECHNICAL QUESTION BANK (EXAMINER "TRAP" QUESTIONS)

Examiners love testing depth with edge cases and unexpected scenarios. Be ready with these defenses:

Q17: What happens if the university campus internet goes down completely? Thapelo / Brown

Answer: "As documented in Phase 3 Section 3.6, UPBS incorporates a **Multi-Channel Fallback Architecture**. Our `Notification` entity supports cellular SMS dispatch. If data connectivity fails, the mobile client triggers an SMS alert containing GPS coordinates and user ID directly to the security base station modem, ensuring distress calls succeed even with zero internet."

Q18: What if two officers click 'Acknowledge' or 'Dispatch' on the same alert at the exact same second? Brown / Thapelo

Answer: "Our database schema explicitly anticipates this! In `AlertResponse`, we define `unique_together = ('alert', 'officer')`. In `api_update_alert_status`, we use `AlertResponse.objects.get_or_create()`. Both officers are recorded as responding units without primary key collisions, and the CAD broadcasts both officers' badges to all units."

Q19: How do you prevent a malicious student from triggering a Denial-of-Service (DoS) by spamming panic? Brown / Palesa

Answer: "1) Identity accountability: every alert is tied to a verified `institutional_id`; 2) State lock: in `panic.js`, the UI disables the panic button while an alert is already active; 3) Backend check: before creating a new alert, the backend verifies if the user already has an unresolved alert created within the last 60 seconds, preventing duplicate spam."

Q20: What happens if an officer closes their browser while an emergency alert is in progress? Rethabile

Answer: "Nothing is lost! The system is completely stateless on the client side. All emergency state is stored in the database. When the officer reopens the browser or another officer logs in from another terminal, the 2.5-second polling stream immediately retrieves all active unresolved alerts from `/api/alerts/active/` and resumes radar tracking."

Q21: How do you ensure privacy for sensitive incidents like Gender-Based Violence (GBV)? Thapelo / Palesa

Answer: "In `IncidentReport`, reports are classified under categories like `HARASSMENT / GBV`. These records are restricted by RBAC so only authorized senior security officers and system administrators can view them. Standard security officers on patrol duty do not see private incident case notes unless assigned to investigate."

Q22: What was the most difficult technical bug your team encountered, and how did you resolve it? Any Member

Answer: "During testing, we discovered that mobile browsers throttled background geolocation updates and blocked autoplay audio for the security siren due to modern browser autoplay policies. We solved the audio issue by tying the `AudioContext` initialization to the officer's first user interaction click (e.g. logging in or clicking 'Enable Sound'), and solved geolocation by adding our native Android Java bridge which runs hardware location listeners independently of browser throttling."

Q23: Why didn't you follow the Java prescription mentioned in slide 4 of the briefing? Brown / Thapelo

Answer: "We actually **did** implement Java! We built a full native Android mobile application in Java located in `android_student_portal/src/main/java/za/ac/spu/upbs/student/MainActivity.java` and compiled it into a signed release APK (`UPBS-Student-Portal.apk`). For the backend, Django was chosen to ensure maximum web security, robust authentication, and relational integrity as approved in our Phase 3 design document."

Q24: If you had 3 more months to develop UPBS, what would you prioritize? All Members

Answer: "1) Integration of physical LoRaWAN IoT panic buttons for outdoor campus paths without cellular reception; 2) Automated SMS gateway dispatch via Twilio / Africa's Talking; 3) Indoor floor-level positioning using BLE (Bluetooth Low Energy) beacons to pinpoint the exact floor and room number inside multi-story campus buildings."

7. PRE-ASSESSMENT CHECKLIST & LIVE DEMO EXECUTION SHEET

Follow this exact operational checklist starting at 07:45 tomorrow morning:

Time Window	Required Operational Action	Responsible Member
07:45 – 08:00	Arrive at Boardroom C006, 3rd Floor. Gather all 4 team members. Ensure everyone is professionally dressed and calm.	All Members
08:00 – 08:15	**Register Slot:** Brown registers 'Techno Warriors' for the earliest available morning assessment slot on the registration sheet.	Brown Nkosi
08:15 – 08:25	**Demonstration Setup & System Pre-Warming:** 1. Plug laptop into power; connect to stable Wi-Fi or phone hotspot. 2. In terminal, seed clean data: `python manage.py seed_upbs`. 2. In terminal, seed clean demo accounts: `python manage.py seed_upbs --demo`. 3. Launch development server: `python manage.py runserver 0.0.0.0:8000`. 4. Open Browser Window 1: `http://127.0.0.1:8000/login/` (Log in as `student1`). 5. Open Browser Window 2 (Incognito): `http://127.0.0.1:8000/login/` (Log in as `officer1`). 6. Open second terminal tab with command typed and ready: `python manage.py test tests`. 6. Open Browser Window 3: `http://127.0.0.1:8000/admin-portal/login/` (Log in as `admin`). 7. Open second terminal tab with command typed and ready: `python manage.py test`.	Thapelo & Rethabile
Call-in	Enter boardroom with confidence. Smile, greet Dr Silas politely. Keep demo pace crisp and articulate.	All Members

PRE-SEEDED DEMO CREDENTIALS CHEAT SHEET

PRE-SEEDED DEMO CREDENTIALS CHEAT SHEET

System Role	Username	Username / ID	Password	Landing Interface	Core Features to Demonstrate to Dr Silas
STUDENT 1	`student1` (`202474004`)	`password123`	`/home/`	Pulsing SOS trigger (Iteration 1), 3s abort window, GPS status pill, 4-stage tracking timeline (Iteration 3).	Pulsing 216px SOS trigger (Iteration 1), 3s audible abort window, GPS status pill, 4-stage tracking timeline (Iteration 3).
STUDENT 2	`student2` (`202437528`)	`password123`	`/home/`	Secondary student account for multi-alert concurrency demonstration.	Secondary student account for multi-alert concurrency demonstration and incident submissions.
SECURITY OFFICER	`officer1` (`SEC001`)	`password123`	`/security/dashboard/`	Live Leaflet GIS radar, audio alarm siren, case claiming, unit dispatch, Haversine distance, incident resolution notes.	Live Leaflet GIS radar, in-memory Web Audio siren, case claiming, unit dispatch, Haversine distance, incident resolution notes.
ADMINISTRATOR	`admin` (`ADMIN001`)	MASTER ADMIN	`admin` (`mnnhlnkosi@gmail.com`)	`password123`	`/analytics/reports/`	Campus emergency KPI tiles, resolution rates, CSV report download, user role management panel.	`/admin-portal/login/` → `/admin-portal/`	Live PostgreSQL query latency telemetry, Render keepalive pulse, live CAD feed, IAM directory, instant user verification & link copy.

EMERGENCY CONTINGENCY PROTOCOLS (IF HARDWARE / NETWORK GLITCHES)

EMERGENCY CONTINGENCY PROTOCOLS (IF HARDWARE / NETWORK GLITCHES)

> **1. If Local Campus Wi-Fi drops:** Immediately enable phone 4G/5G mobile hotspot. Or navigate directly to our live production cloud deployment on Render: **`https://youthalliance.co.za`**. It is connected to Supabase PostgreSQL and runs the identical codebase.

2. If the local server stops: Simply restart with `python manage.py runserver 0.0.0.0:8000`. The database persists all state.

3. Final Words of Advice: Speak clearly and with conviction. Support your teammates. When Dr Silas asks a question to your teammate, do not interrupt; let them deliver their answer. Be proud of what you built—this is an extraordinary, production-grade system!
